

%md

This assignment focused on understanding and implementing **incremental data processing using Delta Lake** in Databricks. The objective was to load an existing dataset into a Delta table, perform data cleaning, simulate incremental data, apply the Delta Lake MERGE operation, and validate the final output.

I started by loading the **customer_master** dataset into a Spark DataFrame using the existing Delta table and displayed the dataset to understand its structure. The dataset contained **9,994 records** and **21 columns**. I verified the schema using `printSchema()` and previewed the first few records to understand the available attributes such as Customer ID, Customer Name, Product details, Sales, Quantity, Discount, and Profit.

The next step was data cleaning. I handled missing values by replacing all null values with **"Unknown"** using the `fillna()` function. Although the dataset did not contain significant missing values, this step ensured consistency in the data. I then removed duplicate records using `dropDuplicates()` and compared the row count before and after cleaning.

Original number of records: 9,994

Number of records after cleaning: 9,994

Since both counts remained the same, it confirmed that the dataset did not contain duplicate records.

While converting the cleaned data into a Delta table, I encountered an issue because Delta Lake does not allow spaces in column names by default. To resolve this, I renamed all columns by replacing spaces with underscores using `withColumnRenamed()`. For example:

``Row ID` → `Row_ID``

``Order ID` → `Order_ID``

``Order Date` → `Order_Date``

``Ship Date` → `Ship_Date``

- * `Customer ID` → `Customer\_ID`
- * `Customer Name` → `Customer\_Name`
- * `Postal Code` → `Postal\_Code`
- * `Product ID` → `Product\_ID`
- * `Product Name` → `Product\_Name`
- * `Sub-Category` → `Sub\_Category`

This made the schema compatible with Delta Lake. The cleaned dataset was then saved as a Delta table named **customer_master_clean**.

To simulate incremental processing, I created a second dataset named **customer_incremental**. I selected the first three records from the master dataset and modified their **Sales** values by increasing them, representing updated transactions for existing customers. I also selected the last two records from the master dataset and modified their **Row_ID**, **Customer_ID**, and **Customer_Name** to simulate two completely new customer records. These updated and new records were combined using the `union()` operation to form the incremental dataset.

The incremental dataset was saved as another Delta table named **customer_incremental** and verified by displaying its contents.

For incremental processing, I created temporary views for both Delta tables and applied the **MERGE** operation. During the merge:

- * Existing records were identified using the **Row_ID** column.
- * Matching records were updated by modifying the **Customer_Name**, **Sales**, and **Profit** columns.
- * Records that did not exist in the master table were inserted as new records using the `WHEN NOT MATCHED THEN INSERT` clause.

This demonstrated the Delta Lake capability of performing update and insert operations within a single transaction.

After completing the MERGE operation, I validated the final output by loading the updated Delta table. I verified the total number of records and checked for duplicate **Row_ID** values using aggregation. No duplicate Row_ID values were found, confirming that the merge operation maintained data integrity.

Finally, I displayed the updated Delta table to verify that both updated and newly inserted customer records were present. This confirmed that the incremental processing workflow had executed successfully.

Overall, this assignment helped me understand the practical implementation of Delta Lake for incremental data processing. I learned how to clean data, resolve schema compatibility issues, create Delta tables, simulate incremental data, perform MERGE operations, and validate the final dataset. The assignment also provided hands-on experience with PySpark DataFrames, Delta Lake storage format, and SQL-based MERGE operations in Databricks, giving me a clear understanding of how incremental data pipelines are implemented in real-world data engineering workflows.